bruise_colab_inference_v2.ipynb — Cell-by-Cell Explanation

What this notebook does. Runs the 5 trained models (SegFormer B2/B0×2, YOLO×2) on the 185-image test set, entirely at 640 × 640. All 5 models share *one* pipeline: a 640 stretch cache, tensors staged on the GPU, per-model rescaling, one speed benchmark over all 185 images, and a val-swept threshold for YOLO. It reuses the `pipeline/` package (no re-upload) and does its own inference/timing inline.

Section 1 — Setup (cells 0–8)

- **Cell 0 (md):** Title + v2 changelog — one benchmark over all 185 images, YOLO threshold swept on val instead of argmax.
- **Cell 2:** Mount Google Drive; check the package zip exists.
- **Cell 4:** Assert a GPU is present (`cuda:0`); timing is meaningless on CPU.
- **Cell 6:** Copy the zip off Drive to local disk and unzip (network latency would pollute timing).
- **Cell 8:** `pip install` transformers, ultralytics, albumentations, opencv (torch is preinstalled on Colab).

Section 2 — Data + models (cells 10–16)

- **Cell 10:** Imports + config. Sets `IMG_H=IMG_W=640` and `cudnn.benchmark=True` (autotune conv kernels for the fixed 640 shape).
- **Cell 12:** *Load test set + build the 640 cache + stage on GPU.* Loads the 185-image test manifest, builds a one-time 640 × 640 PNG cache (image bilinear, mask nearest), then stacks all 185 images onto the GPU as `X_TEST_GPU` (/255, ≈0.9 GB); ground truth `GT_640` stays on CPU. This is the per_model_batch/final dataloader design: neutral /255 loader, resize done once.
- **Cell 14:** Load the 5 trained models as raw `nn.Modules`. SegFormer via `load_segformer_model` (returns its val-calibrated threshold); YOLO via `copy.deepcopy(YOLO(best).model)` — the raw network, not the `.predict()` wrapper.
- **Cell 16:** *Preprocess/postprocess, spelled out.* Defines `_MEAN_T/_STD_T` (ImageNet stats as GPU tensors), and the mask functions. SegFormer = `sigmoid(logit)`≥`threshold`. YOLO bruise prob = `sigmoid`$(z_1 - z_0)$ with an `F.interpolate` upsample 80 → 640; `postprocess_yolo_prob` thresholds it. **Key rule:** the loader is /255; SegFormer rescales to ImageNet, *YOLO stays* /255 (ImageNet norm would collapse YOLO to ∼0.48 Dice).

Section 3 — YOLO val threshold sweep (cells 17–18)

- **Cell 17 (md):** Why: SegFormer has a val-calibrated threshold, so give YOLO one too instead of bare argmax.
- **Cell 18:** Loads the 134 val images (subject-disjoint from test — leak-guarded), stages them /255 on the GPU, and sweeps `sigmoid`$(z_1 - z_0)$≥`thr` over 91 thresholds. Picks the threshold with the best *val* Dice per YOLO model and stores it. Fit on val, applied once to test — no leakage. (argmax is just the special case thr= 0.5.)

Section 4 — Speed benchmark, all 185 (cells 19–22)

- **Cell 19–21 (md):** Explains GPU-correct timing (`cuda.synchronize` around each call; warmup) and the single benchmark. **Included:** forward pass, upsample, threshold. **Excluded:** disk read, resize, host→GPU copy, and the device→host copy (mask stays on GPU).
- **Cell 20:** `benchmark_forward` — times 640 tensor → 640 mask for *all 185* staged images, 3 repeats each (= 555 calls), one image at a time (real single-frame latency). Matches `benchmark_speed` in per_model_batch/final. Returns median / p95 / FPS / peak memory.
- **Cell 22:** Runs the benchmark over all 5 models, prints per-model latency.

Section 5 — Accuracy + save (cells 23–28)

- **Cell 24:** *Accuracy over all 185 staged images.* For each model: SegFormer → `(x−MEAN)/STD` → model → `sigmoid≥threshold`; YOLO → model on /255 → `sigmoid(`$z_1 - z_0$`)≥` *swept* threshold. Scores Dice/IoU/complete-miss vs `GT_640` at 640. No native `.predict()`, no letterbox — one grid for all 5.
- **Cell 26:** Final table — params, threshold, median/mean Dice, IoU, complete-miss%, median/p95 ms, FPS. (Read complete-miss next to Dice: best Dice ≠ safest model.)
- **Cell 28:** Saves per-image CSVs, the results table, `benchmark_all185.csv`, and `metadata.json` (records the sweep, single benchmark, and that the original notebook is kept for comparison) to Drive.

How v2 differs from the original

- YOLO uses the *same dataloader* (stretch 640, /255) as SegFormer — no letterbox, no native `.predict()`.
- 640 cache + all 185 images staged on the GPU once.
- Benchmarks 2 & 3 removed; one benchmark over all 185 (was: 1 image ×100).
- YOLO threshold *swept on val* (symmetric with SegFormer) instead of argmax.